



UDEM

UNIVERSIDAD DE MONTERREY

Práctica 4

Microprocesadores
M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo García Mat: 612348

Nombre: Ricardo Garza Benítez Mat: 645972

"Damos nuestra palabra de que hemos hecho esta actividad con integridad académica"

Monterrey, N. L., a 8 de Junio del 2026

Introducción

El PIC16F887 es un microcontrolador de 8 bits fabricado por Microchip Technology que integra, en un solo encapsulado DIP de 40 pines, múltiples puertos de entrada y salida de propósito general, convertidores analógico-digitales, temporizadores y una arquitectura RISC capaz de ejecutar la mayoría de sus instrucciones en un solo ciclo de máquina. Su organización interna se basa en registros especiales de función (SFR) que permiten configurar individualmente el comportamiento de cada pin mediante registros como TRIS, PORT, ANSEL y ANSELH, lo que lo convierte en una plataforma versátil para el desarrollo de sistemas embebidos de mediana complejidad. [1]

En esta práctica se exploran dos aspectos fundamentales de la programación del PIC16F887: la configuración de puertos digitales y la lectura de entradas externas mediante botones conectados al Puerto B. En la primera actividad se comprueba el funcionamiento de los registros ANSEL y TRIS para convertir pines analógicos en salidas digitales capaces de controlar un display de siete segmentos. En la segunda actividad se implementa un contador bidireccional de dos dígitos, desplegado en dos displays conectados a los puertos C y D, con tres botones que permiten incrementar, decrementar y cambiar el paso de conteo entre uno y dos unidades. [2]

El cristal de cuarzo de 16 MHz establece la frecuencia base de operación del sistema, y mediante la directiva `#define _XTAL_FREQ 16000000` el compilador puede calcular los retardos software correctamente. La detección de flancos descendentes en los botones, implementada mediante comparación con el estado anterior de cada pin, garantiza que cada pulsación sea registrada una sola vez independientemente del tiempo que el botón permanezca presionado, resolviendo el problema de rebote de una forma sencilla y efectiva dentro del contexto de esta práctica. [1]

Marco Teórico

Puertos digitales y registros TRIS, ANSEL y ANSELH

El PIC16F887 posee cinco puertos de entrada/salida etiquetados como A, B, C, D y E. Cada puerto tiene asociado un registro TRIS que controla la dirección de cada pin de forma individual: escribir un '1' en un bit del registro TRIS configura el pin correspondiente como entrada, mientras que escribir un '0' lo configura como salida. Sin embargo, muchos de los pines del Puerto A y del Puerto E están compartidos con el convertidor analógico-digital interno, lo que significa que al encender el microcontrolador quedan habilitados por defecto como entradas analógicas. Para poder usarlos como entradas o salidas digitales es indispensable deshabilitar la función analógica mediante los registros ANSEL y ANSELH, escribiendo un cero en los bits correspondientes a los pines que se

deseen usar en modo digital. Omitir este paso es uno de los errores más comunes al programar este microcontrolador, ya que el pin físicamente responde pero el registro PORT no refleja el nivel lógico correcto. [1]

Display de siete segmentos

Un display de siete segmentos es un dispositivo de visualización formado por siete LEDs independientes, etiquetados convencionalmente como a, b, c, d, e, f y g, más un punto decimal opcional. Cada segmento puede encenderse o apagarse de forma individual para representar dígitos numéricos y algunos caracteres alfanuméricos. Existen dos configuraciones eléctricas: ánodo común, donde todos los ánodos comparten un nodo de alimentación y el encendido se logra llevando el cátodo a nivel bajo; y cátodo común, donde los cátodos comparten tierra y el encendido requiere nivel alto en el ánodo. En esta práctica se emplean displays de cátodo común, por lo que cada byte enviado al puerto activa los segmentos correspondientes mediante nivel lógico alto en los bits asociados. La tabla de decodificación utilizada en el código asigna un valor hexadecimal específico a cada dígito: por ejemplo, 0xFC representa el cero, encendiendo los segmentos a, b, c, d, e y f; y 0x60 representa el uno, encendiendo únicamente b y c. [2]

Detección de botones y manejo de rebote por software

Los botones mecánicos producen un fenómeno conocido como rebote (bouncing): en el instante de la transición, los contactos abren y cierran múltiples veces en un intervalo de microsegundos antes de estabilizarse. Cuando el microcontrolador lee el estado del pin a alta velocidad puede interpretar cada uno de esos rebotes como una pulsación independiente, generando conteos erróneos. Una solución de software ampliamente utilizada consiste en detectar únicamente el flanco de bajada de la señal, comparando el estado actual del pin con el estado almacenado en la iteración anterior del ciclo principal. Solo cuando el estado previo era alto (botón sin presionar) y el estado actual es bajo (botón presionado) se considera que ocurrió una pulsación válida. Para complementar esta detección y asegurar que el estado del pin se haya estabilizado antes de ser leído, se incluye un retardo de algunos milisegundos al final de cada ciclo del bucle principal mediante la función `__delay_ms()`. [1]

Contador modular bidireccional

Un contador modular es aquel cuya secuencia de valores tiene un límite inferior y uno superior, volviendo al extremo opuesto al desbordarse. En esta práctica el contador opera en el rango de 0 a 99, con aritmética modular implementada directamente en código: al superar 99 se resta 100 para reiniciar al inicio del rango, y al caer por debajo de 0 se suman 100 para volver al valor máximo. Esta lógica permite que la variable contador sea de tipo `signed char`, lo que facilita la detección del desbordamiento negativo

sin necesidad de comparaciones adicionales. El paso variable, controlado por el tercer botón, alterna entre 1 y 2, de modo que el mismo mecanismo de incremento y decremento opera correctamente para ambos modos sin duplicar la lógica de detección de botones. [2]

Simulación

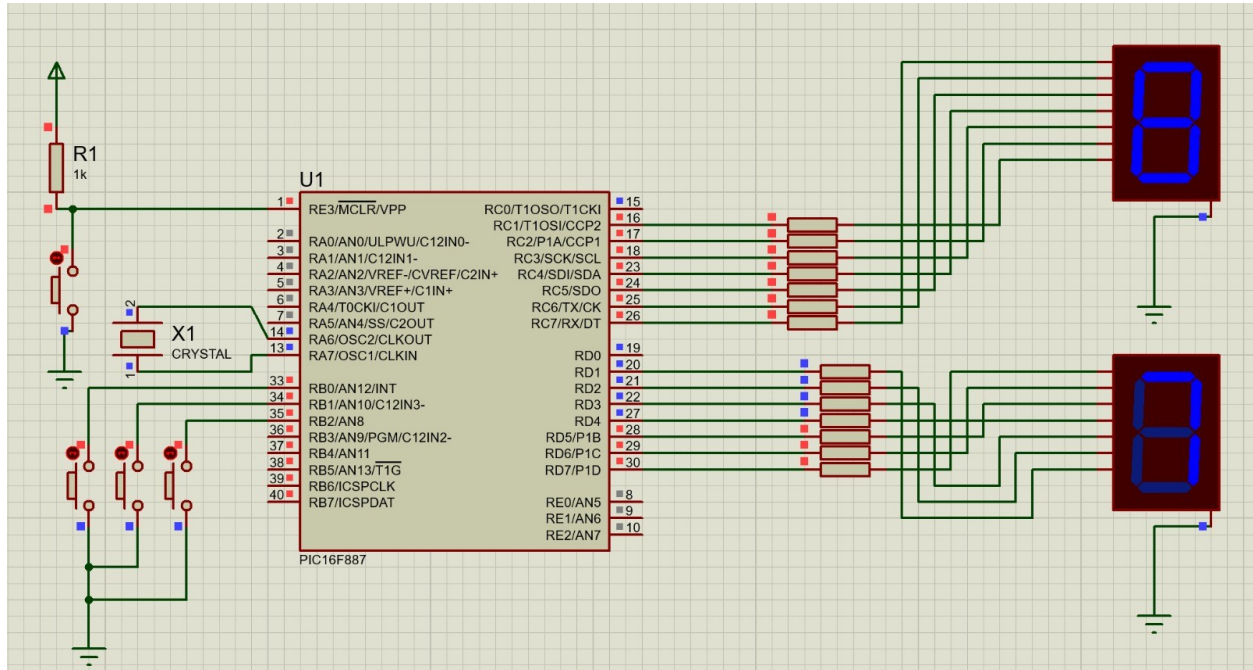
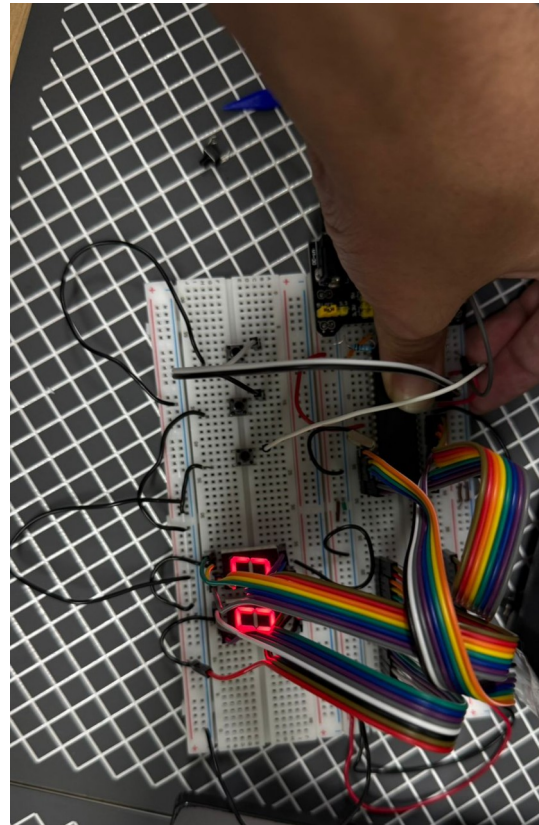
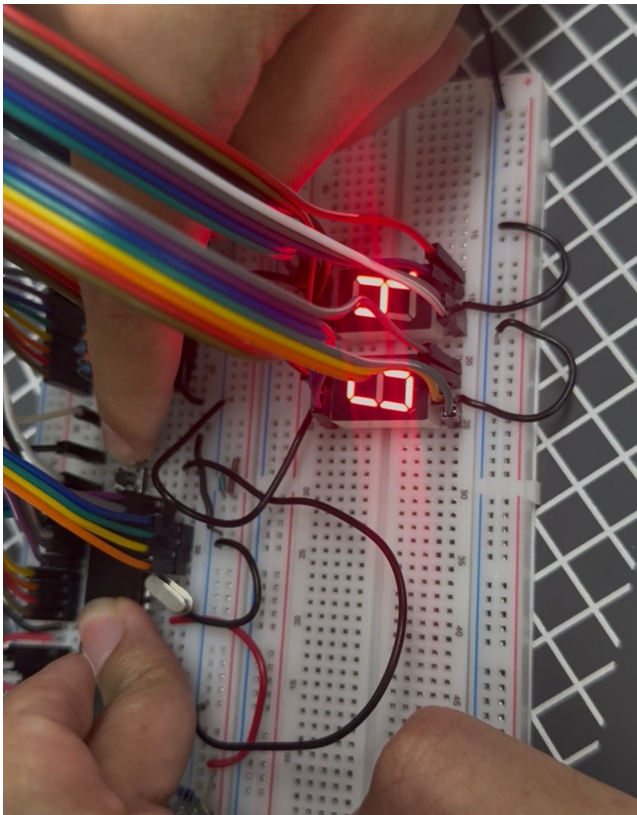


Foto del circuito en Físico



Resultados

Actividad 1 — Configuración de Puerto B como entrada digital

Durante la primera actividad se comprobó de forma directa la importancia de los registros ANSEL y ANSELH para habilitar el comportamiento digital de los puertos del PIC16F887. Al configurar $ANSEL = 0x00$ y $ANSELH = 0x00$, todos los pines que de fábrica se inicializan como entradas analógicas quedan disponibles para operar en modo digital. A partir de esta configuración y con $TRISD = 0x00$, el Puerto D funcionó correctamente como salida, enviando los valores de la tabla de decodificación al display de siete segmentos. Se observó que el ciclo secuencial de 0 a F se desplegó con un intervalo de 200 ms entre cada dígito, recorriendo los dieciséis caracteres hexadecimales de forma continua y sin errores de visualización.

La práctica dejó en evidencia que la inicialización correcta de los registros de configuración es un paso previo obligatorio antes de cualquier operación sobre los puertos. En particular, al verificar el impacto de omitir la configuración de ANSEL, se confirmó que el pin no responde como salida digital si la función analógica permanece habilitada, lo cual refuerza la importancia de revisar el datasheet del dispositivo antes de asumir el comportamiento por defecto de cada pin. [1]

Actividad 2 — Contador bidireccional 0–99 con paso variable

En la segunda actividad se implementó un contador de dos dígitos desplegado simultáneamente en los puertos C y D, representando las decenas y las unidades respectivamente. Los tres botones conectados al Puerto B respondieron correctamente a las operaciones asignadas: el primer botón incrementó el contador según el paso activo, el segundo lo decrementó, y el tercero alternó el paso entre 1 y 2. La detección por flanco de bajada, comparando el estado actual de cada pin con su valor en el ciclo anterior, garantizó que cada pulsación generara exactamente un cambio en el contador, sin lecturas duplicadas. La aritmética modular funcionó correctamente en ambos extremos del rango, saltando de 99 a 0 al incrementar y de 0 a 99 al decrementar.

Durante las pruebas físicas se observó que al pulsar los botones de forma muy rápida y repetida, el contador dejaba de responder correctamente en algunas ocasiones. Este comportamiento se originaba por la ausencia de un retardo entre lecturas, que provocaba que el microcontrolador registrara el mismo flanco múltiples veces o que el estado del pin no se hubiera estabilizado al momento de la lectura. La solución fue agregar un retardo de 5 ms al final del ciclo principal mediante `__delay_ms(5)`, lo que proporcionó el tiempo suficiente para que los contactos del botón se estabilizaran y para que el cambio de estado anterior fuera actualizado antes de la siguiente lectura. Con este ajuste, el contador respondió de forma confiable incluso durante pulsaciones rápidas y sostenidas.

El comportamiento del sistema en modo de paso 2 fue consistente con lo esperado: los incrementos y decrementos se realizaron de dos en dos manteniendo la aritmética modular funcional, y el tercer botón permitió alternar entre modos en cualquier punto de la secuencia sin reiniciar el contador ni alterar su estado. En conjunto, los resultados de ambas actividades confirman que la configuración correcta de los registros de dirección y la implementación de un mecanismo básico de antirrebote son suficientes para construir un sistema de interfaz con el usuario funcional y robusto sobre el PIC16F887.

Conclusiones Individuales

Ricardo Garza:

Esta práctica me dejó una lección muy concreta sobre la diferencia entre saber que un pin existe y saber cómo configurarlo correctamente. Al armar el circuito físico de la primera actividad me di cuenta de que el display no respondía como esperaba, y al revisar el código con más cuidado encontramos que los registros ANSEL no estaban siendo inicializados de manera adecuada para todos los puertos. Ver físicamente cómo un pin que aparece configurado como salida en el TRIS no funciona como salida digital

si la función analógica sigue activa fue una de esas experiencias que fijan el conocimiento de una manera que ninguna explicación teórica logra completamente. Para la segunda actividad, el problema con el rebote de botones al presionarlos repetidamente fue igualmente revelador: el código era correcto en lógica, pero sin el retardo adecuado la velocidad del microcontrolador trabajaba en su contra. Agregar los 5 ms de delay resolvió el problema de inmediato, y eso me hizo entender que en sistemas embebidos el tiempo de respuesta del hardware y el tiempo de ciclo del software tienen que estar coordinados. Me llevo la idea de que revisar paso a paso, primero la configuración de registros y luego el comportamiento esperado contra el observado, es la forma más eficiente de depurar este tipo de sistemas.

Eugenio Romo:

En esta práctica pude comprender de manera más completa cómo se estructura la comunicación entre el software y el hardware en un microcontrolador como el PIC16F887. La primera actividad dejó claro que la configuración de un puerto no se limita a indicar si es entrada o salida con el registro TRIS, sino que requiere también deshabilitar las funciones analógicas a través de ANSEL y ANSELH cuando se trabaja con pines que tienen múltiples funciones asignadas. Este detalle, que parece menor al leer el código, tiene consecuencias directas en el funcionamiento del circuito. En la segunda actividad resultó interesante observar cómo la detección de flancos en lugar de niveles permite un control mucho más preciso sobre la respuesta a los botones, y cómo un retardo de apenas unos milisegundos es suficiente para hacer que el sistema se comporte de manera confiable. La práctica reforzó la idea de que en el diseño embebido, la correcta inicialización del hardware y el manejo del tiempo son aspectos tan importantes como la lógica del programa en sí.

Referencias:

[1] Microchip Technology. (2009). PIC16F887 Data Sheet: 28/40/44-Pin Enhanced Flash Microcontrollers. Microchip Technology Inc.

<https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>

[2] Floyd, T. L. (2006). Fundamentos de sistemas digitales (9.^a ed.). Pearson Educación.

https://gc.scalahed.com/recursos/files/r161r/w25733w/fundamentos-de-sistemas_compressed.pdf